INVENTORY & SALES MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

Roll Number	Registration No.	Student Code	Student Name
26102410033	26102000155	BWU/MCA/26/037	DEBARGHA BISWAS

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

IN

COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY

398, Ramkrishnapur Rd, Barasat, North 24 Paraganas, Kolkata, West Bengal 700125

December, 2026

INVENTORY & SALES MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

Roll Number	Registration No.	Student Code	Student Name
26102410033	26102000155	BWU/MCA/26/037	DEBARGHA BISWAS

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

IN

COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY

398, Ramkrishnapur Rd, Barasat, North 24 Paraganas, Kolkata, West Bengal 700125

ACKNOWLEDGEMENT

The successful completion of the ”Inventory & Sales Management System” project is the culmination of dedication, hard work, and the unwavering support of many individuals and institutions.

I wish to express my deepest gratitude to all those who contributed to making this endeavor a success.

First and foremost, I extend my profound thanks to my esteemed Project Mentor, Dr. Rahul Kumar Ghosh, for their exceptional guidance, insightful critiques, and constant encouragement throughout overcoming complex challenges.

I am immensely grateful of the BRAINWARE UNIVERSITY for providing the necessary resources, advanced facilities, and an encouraging academic environment that fostered the technical skills required for this project.

I would also like to acknowledge the support of Dr. Jayanta Aich, Head of the Computational Sciences Department, for their administrative support and facilitation of resources.

Finally, I owe a special debt of gratitude to my family and friends, whose endless patience, belief, and moral support provided the motivation to persevere and successfully conclude this project.

Project Members:

2. BWU/MCA/26/037 - **DEBARGHA BISWAS**

Date: _____

Department of Computational Sciences
Brainware University

ABSTRACT

Managing inventory and daily sales manually can be disorganized and prone to errors, particularly for small businesses with limited resources. This project presents the development of a streamlined, Command Line Interface (CLI) Inventory and Sales Management System designed to digitize and simplify these routine operations. Built using Python and Object-Oriented Programming principles, the software provides a lightweight and highly responsive environment that runs directly in a standard computer terminal. The system automates crucial tasks such as adding new stock, processing customer transactions, enforcing low-stock alerts, and generating unique receipts, all while instantly updating inventory quantities to prevent accidental stockouts. To ensure reliable data storage, the application utilizes the Pandas library to seamlessly save and retrieve records using CSV files, preventing any data loss between sessions. Additionally, the system features built-in data analysis tools powered by Matplotlib, which transform raw sales transaction numbers into visual charts and graphs for better business insights. Ultimately, this project delivers a fast, cost-effective, and user-friendly digital solution that enhances operational efficiency without requiring expensive hardware or complicated graphical software setups.

Keywords: Inventory Management, Sales Tracking, Command Line Interface (CLI), Python, Object-Oriented Programming, Data Storage, Pandas, Matplotlib.

TABLE OF CONTENTS

Acknowledgement . . . . . **i**

Abstract . . . . . **ii**

1 INTRODUCTION **1**

2 OBJECTIVE **2**

3 PLANNING **3**

4 REQUIREMENT ANALYSIS **4**

5 PROPOSED DESIGN **5**

5.1 Data Storage and Loading 5

5.2 Product Management 6

5.3 Processing Sales 7

5.4 Monitoring and Analysis 7

5.5 Sorting and Organizing Inventory 9

6 Experimental Results **11**

7 Conclusion **16**

REFERENCES **17**

Chapter 1

INTRODUCTION

The Inventory and Sales Management System is a streamlined, Command Line Interface (CLI) application designed to help businesses easily track their products, manage stock levels, and record daily transactions. In many small to medium-sized operations, keeping track of inventory and sales manually using paper records or basic spreadsheets can quickly become disorganized, time-consuming, and prone to human error. This project provides a practical digital solution to that problem by offering a fast, text-based environment where users can handle all their core store operations directly from a terminal window.

Built entirely to run in the command prompt, the system removes the need for complex and heavy graphical interfaces, focusing instead on speed, reliability, and ease of use. Users can seamlessly navigate through straightforward menu options to add new items to their inventory, update quantities when new stock arrives, process customer sales, and instantly view up-to-date records. By automating these essential tasks, the software ensures that stock levels remain accurate at all times, helping business owners avoid situations where popular items run out unexpectedly or unsold items pile up.

Furthermore, the system is designed with simplicity in mind so that anyone with basic computer knowledge can use it without a steep learning curve. Beyond just storing data safely, the application keeps all sales records highly organized, laying the groundwork for better business decisions and data analysis. Ultimately, this project serves as a lightweight, highly efficient tool that brings order and accuracy to daily business management, proving that you do not need overly complicated software to keep an operation running smoothly.

Chapter 2

OBJECTIVE

The primary objective of the Inventory and Sales Management System is to provide small businesses and shop owners with a fast, accurate, and easy-to-use digital tool for handling their daily store operations. The goal is to replace slow, paper-based tracking methods and basic spreadsheets with a reliable computerized system that operates directly from the command line. By doing so, the project aims to give users a lightweight application that runs smoothly on almost any standard computer without requiring expensive hardware or complicated software installations.

A major focus of this project is to eliminate human errors that often happen when counting stock or calculating totals manually. The system is designed to automatically update stock quantities the moment a sale is processed or new items are added. This ensures that the inventory records are always completely accurate and up to date. Another key objective is to help business owners avoid running out of popular items by giving them instant access to their current stock levels. Alongside inventory tracking, the project aims to securely record all daily sales transactions in an organized way, meaning users will never have to dig through piles of paper receipts to check their past sales.

Ultimately, the objective is to build a software tool that is both highly functional and incredibly simple to navigate. Because it is entirely text-based, the design focuses on clear menus and straightforward prompts. The goal is to ensure that anyone, regardless of their computer experience, can learn to add items, process sales, and check inventory with minimal training. By keeping the design simple and the performance fast, the project strives to make daily business management less stressful and much more efficient.

Chapter 3

PLANNING

The planning phase of the Inventory and Sales Management System focused on breaking down the project into clear, manageable steps before any coding began. The first step was to identify exactly what the system needed to accomplish to be truly useful for a small business. We decided that the core features must include adding new products, updating stock quantities, processing customer sales, and viewing the current inventory status. Once these essential features were outlined, the next step was to choose the right environment. Since the primary goal was to keep the application fast, lightweight, and easy to run on any computer, we planned to build a Command Line Interface (CLI) rather than a complicated graphical application.

After deciding on the text-based interface, we planned how the system would handle and store data. We determined that the application needed a simple but reliable way to save inventory details and sales records so that no information would be lost when the program was closed. We then mapped out the basic text menus to ensure the user experience would be logical and easy to navigate. We planned for a main menu that would act as a central hub, where each option selected by the user would trigger a specific function in the background. This modular approach was chosen to keep the underlying code organized and much easier to troubleshoot or upgrade later on.

Finally, we established a step-by-step timeline for the actual development process. The plan was to first build the foundation for storing data, then create the functions for managing inventory, and follow up with the sales processing features. Once the core functions were ready, we planned to tie everything together with the interactive user menu. We also dedicated a significant portion of the planning phase to testing, ensuring we would have time to check that all mathematical calculations were accurate and that the program would not crash if a user accidentally typed in the wrong command. By structuring the project this way, we ensured a smooth, organized development process from start to finish.

Chapter 4

REQUIREMENT ANALYSIS

The requirement analysis phase focused on understanding exactly what the software must do and what resources are needed to make it work properly. We looked at the problem from the perspective of a small business owner to figure out the essential features that would make their daily tasks easier. We determined that the system needed to be completely self-contained and run locally, meaning it should not require a constant internet connection or expensive cloud servers. Instead, it must run smoothly on any standard desktop or laptop using just a basic text terminal or command prompt. By analyzing these needs, we ensured that the final product would be practical and immediately useful for its intended audience.

During this phase, we established the specific tasks the software must be able to perform. First and foremost, it requires a reliable way to read and save data permanently, ensuring that inventory counts and sales records are not lost when the program closes. The system also needs a simple, number-based menu so users can navigate quickly using only their keyboard. Additionally, we identified that the software must handle all basic math automatically, such as deducting sold items from the overall stock and calculating the total bill for a customer. We also recognized a strong need for error protection; if a user accidentally types a letter instead of a number, or tries to sell more items than are currently in stock, the program must display a helpful warning message rather than freezing or crashing.

On the technical side, we intentionally kept the system requirements very low so that anyone could install and use the program without buying new equipment. The hardware requirements consist of any basic computer with minimal memory and storage space, as text-based applications take up almost no room. For the software side, the computer simply needs the necessary programming environment, such as Python, installed to run the code, along with basic file-reading capabilities to store the records. By keeping the technical and operational requirements this straightforward, we ensured that the system remains accessible, reliable, and cheap to maintain for any small business.

Chapter 5

PROPOSED DESIGN

The system is built using Object-Oriented Programming, dividing the application into three main data structures: Product, Sale, and Inventory. Below are the core functions that drive the application, along with their code snippets and simple explanations.

5.1 Data Storage and Loading

The application uses the Pandas library to seamlessly read and write data to CSV files, ensuring no information is lost when the program closes.

```
1
2  def load_data(self):
3      inventory_df = pd.read_csv(INVENTORY_FILE)
4      sales_df = pd.read_csv(SALES_FILE)
5      for row in inventory_df.itertuples(index=False):
6          product_dict = {
7              'product_id' : int(row.product_id),
8              'product_name' : str(row.product_name),
9              'category' : str(row.category),
10             'price' : float(row.price),
11             'quantity' : int(row.quantity)
12         }
13         self.products_collection[row.product_id] = Product(product_dict)
14
15     for row in sales_df.itertuples(index=False):
16         sale_dict = {
17                 'transaction_id':str(row.transaction_id),
18                 'date' : str(row.date),
19                 'product_id' : int(row.product_id),
20                 'quantity_sold' : int(row.quantity_sold),
21                 'total_amount' : float(row.total_amount)
22             }
23         self.sales_records[row.transaction_id] = Sale(sale_dict)
24
25 def save_data(self):
26     inventory_list = [product_obj.get_details() for product_obj in self.products_collection.values()]
27     inventory_df = pd.DataFrame(inventory_list)
28     inventory_df.to_csv(INVENTORY_FILE, index=False)
```

Figure 5.1: Data Storage and Loading Functions

These functions act as a bridge between the software and the computer's hard drive. load_data grabs the saved records from inventory.csv and sales.csv and loads them into memory so the system can read them. Conversely, save_data takes all the new changes made during the session and permanently overwrites the files to keep them updated.

5.2 Product Management

The system provides straightforward methods for introducing new items and updating existing stock.

```python

def add_product(self, product: dict):
    if product['product_id'] in self.products_collection:
        raise InvalidProductIdError(message="Product ID already exist. Use Update Stock function")
    else:
        product_obj = Product(product)
        self.products_collection[product['product_id']] = product_obj
        print("PRODUCT ADDED.")
        print('\n')

def search_product(self, product_id: int):
    if product_id in self.products_collection:
        product_obj = self.products_collection[product_id]
        product_list = [product_obj.get_details()]
        product_df = pd.DataFrame(product_list)
        product_df =product_df.rename(columns={'product_id':'PRODUCT ID', 'product_name':'PRODUCT NAM
E', 'category':'CATEGORY', 'price':'PRICE', 'quantity':'QUANTITY'})
        print("=== Product Details ===")
        print(product_df)
        print('\n')
    else:
        raise InvalidProductIdError(message="Product Not Found.")

def update_product(self, product_id: int, add_quantity: int): #ALL DONE
    if product_id not in self.products_collection:
        raise InvalidProductIdError()
    else:
        if(add_quantity < 0):
            raise ValueError("Quantity cannot be a negative integer")
        else:
            product_obj = self.products_collection[product_id]
            product_obj.add_stock(add_quantity)
            product_list = [product_obj.get_details()]
            product_df = pd.DataFrame(product_list)
            product_df =product_df.rename(columns={'product_id':'PRODUCT ID', 'product_name':'PRODUCT
NAME', 'category':'CATEGORY', 'price':'PRICE', 'quantity':'QUANTITY'})
            print("=== Updated Product Details ===")
            print(product_df)
            print('\n')

```

Figure 5.2: Product Management Functions

The add_product function allows users to register a completely new item into the shop, first double-checking that the ID hasn't been used yet to prevent duplicates. When a new shipment arrives for an item already in the system, update_product simply adds the new amount to the current stock without forcing the user to retype the item's name or price.

5.3 Processing Sales

Sales operations are handled securely, automatically tying inventory deduction to revenue generation.



```
def transaction(self, product_id: int, asked_quantity: int, date_sold: str): # ALL DONE
    if product_id not in self.products_collection:
        raise InvalidProductIdError()
    else:
        if self.products_collection[product_id].quantity < asked_quantity:
            raise InsufficientStockError()
        else:
            product_obj = self.products_collection[product_id]
            product_price = product_obj.price
            total_amount = product_price * asked_quantity
            product_obj.reduce_stock(asked_quantity)
            transaction_id = self.generate_transaction_id()
            transaction_date = date_sold
            sale_obj = Sale(sale={
                                    'transaction_id':transaction_id,
                                    'date':transaction_date,
                                    'product_id':product_obj.product_id,
                                    'quantity_sold':asked_quantity,
                                    'total_amount':total_amount
                                })
            self.sales_records[transaction_id] = sale_obj
            receipt = sale_obj.get_receipt()
            print('####################################')
            print(f"#Sold {receipt['quantity_sold']}x {product_obj.product_name}. Total Bill:Rs.{recei
pt['total_amount']}")
            print('####################################')
            print('\n')
```

Figure 5.3: Processing Sales Functions

The transaction function operates as a digital checkout counter. Before allowing a sale, it checks if the store actually has enough of the item in stock. If approved, it automatically calculates the total customer bill, removes the sold amount from the virtual shelf, generates a random 7-digit receipt number, and securely saves the transaction.

5.4 Monitoring and Analysis

The software goes beyond data entry by providing visual insights into business performance using Matplotlib.



```
def low_stock_products(self, threshold: int): #ALL DONE
    if threshold < 0:
        raise ValueError("Threshold cannot be negative!")
    needs_restock = []
    for prod_obj in self.products_collection.values():
        product_detail = prod_obj.get_details()
        if product_detail['quantity'] <= threshold:
            needs_restock.append([product_detail['product_id'], product_detail['product_nam
e']])
    if needs_restock:
        print("Following products with Product ID needs to be restocked: ", end=' ')
        for product in needs_restock:
            print(product, end=', ')
        print('\n')
    else:
        print("All Stock Levels are healthy")
        print('\n')
def total_sales(self): # ALL DONE
    overall_revenue = 0
    for sale_obj in self.sales_records.values():
        overall_revenue += sale_obj.total_amount # Summing up all the transactions happened
in the sales
    print('###################################')
    print(f"#Total Revenue Generated: Rs.{overall_revenue}")
    print('###################################')
    print('\n')

def generate_reports(self):
    sale_df = pd.read_csv(SALES_FILE)
    inventory_df = pd.read_csv(INVENTORY_FILE)
    if not self.sales_records or sale_df.empty:
        raise Exception("Not enough sales data to generate reports")
    else:
        plt.figure(figsize=(20,20))
        # PRODUCT BAR GRAPH
        plt.subplot(3,3,1)
        grouped_data_prod = sale_df.groupby('product_id', as_index=False)['quantity_sold'].s
um().rename(columns={'quantity_sold':'total_units_sold'})
        plt.bar(grouped_data_prod['product_id'].astype(str), grouped_data_prod['total_units_
sold'])
        plt.title("Total Units Sold per Product")
        plt.xlabel('Product ID')
        plt.ylabel("Total Quantity Sold")
        # CATEGORY PIE GRAPH
        plt.subplot(3,3,5)
        merged_df = pd.merge(sale_df, inventory_df, on='product_id')
        category_group = merged_df.groupby('category')
        category_summary = category_group['total_amount'].sum().reset_index()
        plt.pie(category_summary['total_amount'], labels=category_summary['category'], autop
ct='%1.1f%%')
        plt.title('Sales Distribution by Category')
        # MONTHLY PLOT GRAPH
        plt.subplot(3,3,9)
        sale_df['date'] = pd.to_datetime(sale_df['date'])
        sale_df['month_year'] = sale_df['date'].dt.to_period('M')
        monthly_revenue = sale_df.groupby('month_year')['total_amount'].sum().reset_index()
        monthly_revenue['month_year'] = monthly_revenue['month_year'].astype(str)
        plt.plot(monthly_revenue['month_year'], monthly_revenue['total_amount'], color='#2ca
02c', marker='o', linestyle='-')
        plt.title('Monthly Revenue Trend')
        plt.xlabel('Timeline(Month)')
        plt.ylabel('Total Revenue (Rs.)')

        plt.show()
```

Figure 5.4: Monitoring And Analysis Functions

The low_stock_products function acts as an early warning system, printing out a list of items that fall below a specific quantity limit chosen by the user. Finally, generate_reports

transforms the boring text data into three distinct charts: a bar chart for most sold items, a pie chart breaking down revenue by category, and a line graph tracking sales trends over time.

5.5 Sorting and Organizing Inventory

As a business grows, viewing a massive list of unorganized items can become overwhelming. The system provides a flexible sorting function to help users quickly analyze their stock from different perspectives.



```
def sort_products(self, sort_order: str): # SORTING BY PRICE, QUANTITY DONE (SALE SORTING IS PENDING)
    match sort_order:
        case 'PRICE':
            sortedbyprice_dict = dict(sorted(self.products_collection.items(), key=lambda item: item[1].price))
            inventory_list = [product_obj.get_details() for product_obj in sortedbyprice_dict.values()]
            inventory_df = pd.DataFrame(inventory_list)
            inventory_df = inventory_df.rename(columns={'product_id':'PRODUCT ID', 'product_name':'PRODUCT NAME', 'category':'CATEGORY', 'price':'PRICE', 'quantity':'QUANTITY'})
            print(inventory_df)
            print("\n")
        case 'QUANTITY':
            sortedbyquantity_dict = dict(sorted(self.products_collection.items(), key=lambda item: item[1].quantity))
            inventory_list = [product_obj.get_details() for product_obj in sortedbyquantity_dict.values()]
            inventory_df = pd.DataFrame(inventory_list)
            inventory_df = inventory_df.rename(columns={'product_id':'PRODUCT ID', 'product_name':'PRODUCT NAME', 'category':'CATEGORY', 'price':'PRICE', 'quantity':'QUANTITY'})
            print(inventory_df)
            print("\n")
        case 'SALE':
            sales_by_product = {}
            for sale in self.sales_records.values():
                sales_by_product[sale.product_id] = (sales_by_product.get(sale.product_id, 0) + sale.quantity_sold)
            sorted_products = sorted(self.products_collection.values(), key=lambda product: sales_by_product.get(product.product_id, 0), reverse=True)
            inventory_list = []
            for product in sorted_products:
                details = product.get_details()
                details['quantity_sold'] = sales_by_product.get(product.product_id, 0)
                inventory_list.append(details)
            inventory_df = pd.DataFrame(inventory_list).rename(columns={
                'product_id' : 'PRODUCT ID',
                'product_name' : 'PRODUCT NAME',
                'category' : 'CATEGORY',
                'price' : 'PRICE',
                'quantity' : 'QUANTITY',
                'quantity_sold' : 'QUANTITY SOLD'
            })
            print(inventory_df)
            print("\n")
        case _:
            raise ValueError("Invalid Sorting Order!")
```

Figure 5.5: Sorting and Organizing Functions

The sort_products function acts as a smart filter for the inventory table. It uses a straight-forward match system to reorganize the displayed data based on exactly what the user wants to see.

Chapter 6

Experimental Results

To verify that the Inventory and Sales Management System works reliably under real-world conditions, a series of practical experiments were conducted directly in the command-line interface. These tests simulated a typical day of shop operations to ensure all functions handled data correctly and that the system safely caught any user mistakes without crashing.

The first experiment tested data entry and validation by adding a series of new items to the system. When valid details like Product ID, Name, Category, Price, and Quantity were entered, the system successfully stored them and immediately displayed the updated inventory table. To test the system's error protection, an attempt was made to add a new product using an ID that already existed in the database. As expected, the system blocked the entry and displayed an "InvalidProductIdError," prompting the user to either use a different ID or use the Restock function instead. The Restock function was then tested by adding 20 units to an existing item, which correctly updated the overall quantity without requiring the user to re-enter the item's price or category.

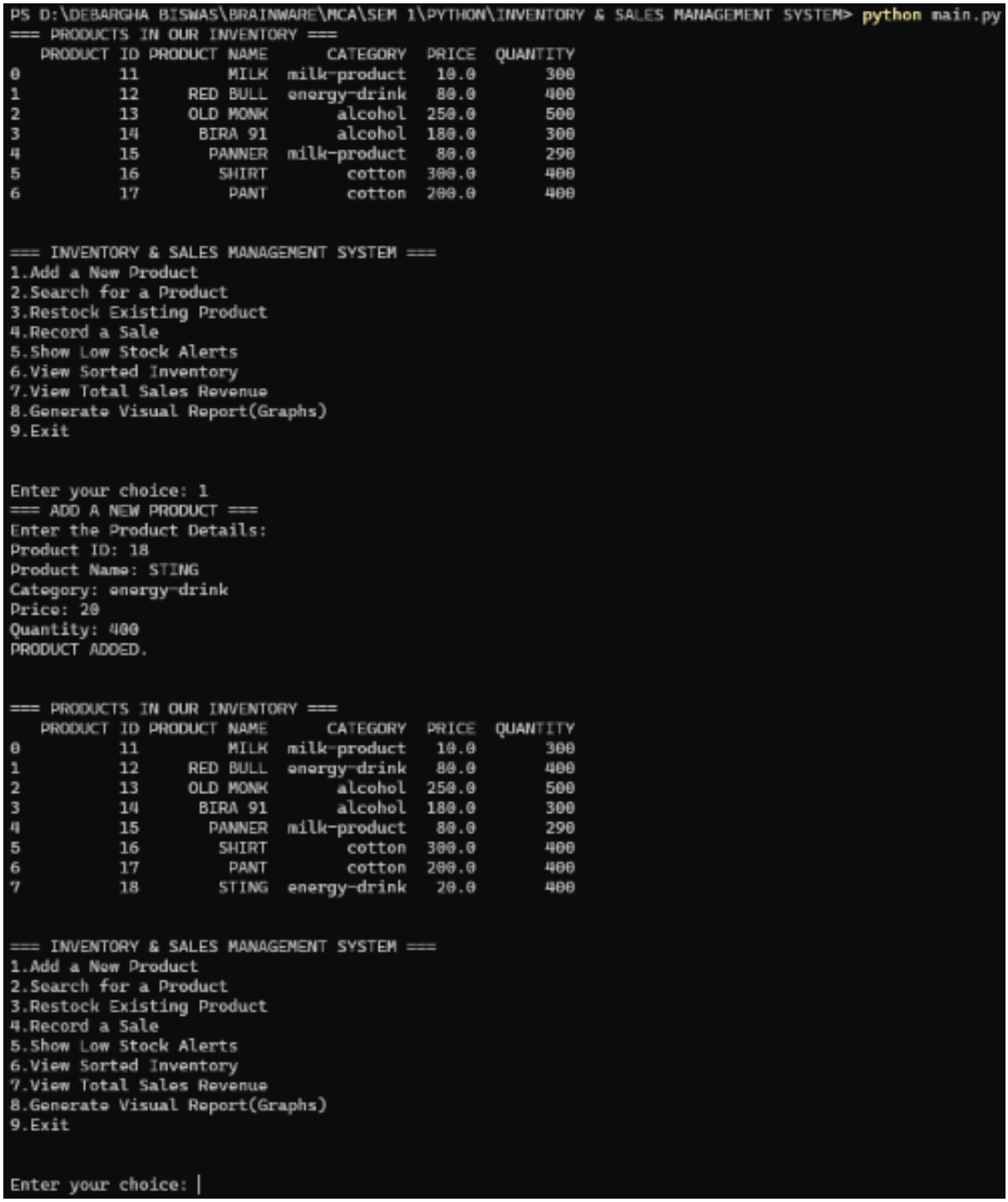


Figure 6.1: Main Interface and Successful Data Entry

The second major experiment focused on transaction processing and automated math calculations. A test sale was recorded by entering an existing product ID and a requested quantity. The software successfully calculated the total bill by multiplying the price by the quantity, generated a unique random 7-digit transaction ID, and printed a formatted receipt to the terminal. Most importantly, when the inventory was viewed immediately after this sale, the stock quantity of that item had automatically decreased by the exact amount sold. We then deliberately

attempted to sell 50 units of an item that only had 10 units in stock. The system successfully caught this impossible request, stopped the transaction, and displayed an ”InsufficientStock-Error,” proving that the inventory cannot be accidentally pushed into negative numbers.

```
=== PRODUCTS IN OUR INVENTORY ===
   PRODUCT ID PRODUCT NAME      CATEGORY  PRICE  QUANTITY
0          11         MILK  milk-product   10.0       300
1          12     RED BULL  energy-drink   80.0       400
2          13     OLD MONK       alcohol  250.0       500
3          14      BIRA 91       alcohol  180.0       300
4          15       PANNER  milk-product   80.0       290
5          16        SHIRT        cotton  300.0       400
6          17         PANT        cotton  200.0       400
7          18        STING  energy-drink   20.0       400


=== INVENTORY & SALES MANAGEMENT SYSTEM ===
1.Add a New Product
2.Search for a Product
3.Restock Existing Product
4.Record a Sale
5.Show Low Stock Alerts
6.View Sorted Inventory
7.View Total Sales Revenue
8.Generate Visual Report(Graphs)
9.Exit


Enter your choice: 4
=== RECORD A SALE ===
Enter Product Id and quantity to make a sale...
Product ID: 17
Quantity: 500
Date Sold(YYYY-MM-DD): 2026-11-02
Application Error: Error: Not enough stock to complete the transaction.
```

Figure 6.2: Error Handling and Preventing Mistakes

```
=== PRODUCTS IN OUR INVENTORY ===
   PRODUCT ID PRODUCT NAME      CATEGORY  PRICE  QUANTITY
0          11         MILK  milk-product   10.0       300
1          12     RED BULL  energy-drink   80.0       400
2          13     OLD MONK       alcohol  250.0       500
3          14      BIRA 91       alcohol  180.0       300
4          15       PANNER  milk-product   80.0       290
5          16        SHIRT        cotton  300.0       400
6          17         PANT        cotton  200.0       400
7          18        STING  energy-drink   20.0       400


=== INVENTORY & SALES MANAGEMENT SYSTEM ===
1.Add a New Product
2.Search for a Product
3.Restock Existing Product
4.Record a Sale
5.Show Low Stock Alerts
6.View Sorted Inventory
7.View Total Sales Revenue
8.Generate Visual Report(Graphs)
9.Exit


Enter your choice: 4
=== RECORD A SALE ===
Enter Product Id and quantity to make a sale...
Product ID: 18
Quantity: 100
Date Sold(YYYY-MM-DD): 2026-11-05
####################################
#Sold 100x STING. Total Bill:Rs.2000.0
####################################
```

Figure 6.3: Sale Receipt

The final experiments tested the system's analytical tools and data saving features. The sorting function was tested by organizing the inventory by "SALE". The system successfully analyzed the entire transaction history and reorganized the terminal display to show the highest-selling items at the very top. Next, the low-stock alert was tested by setting a threshold of 15 units. The system accurately scanned the stock and printed out a warning list of only the items that fell at or below that number. When the visual reports option was selected, the software successfully paused the terminal and launched a new window displaying the bar chart, pie chart, and line graph, proving the Matplotlib integration worked flawlessly. Finally, the exit command was triggered, and the system successfully wrote all the newly added inventory and

sales data back into the ”inventory.csv” and ”sales.csv” files, ensuring zero data loss after the session was closed.

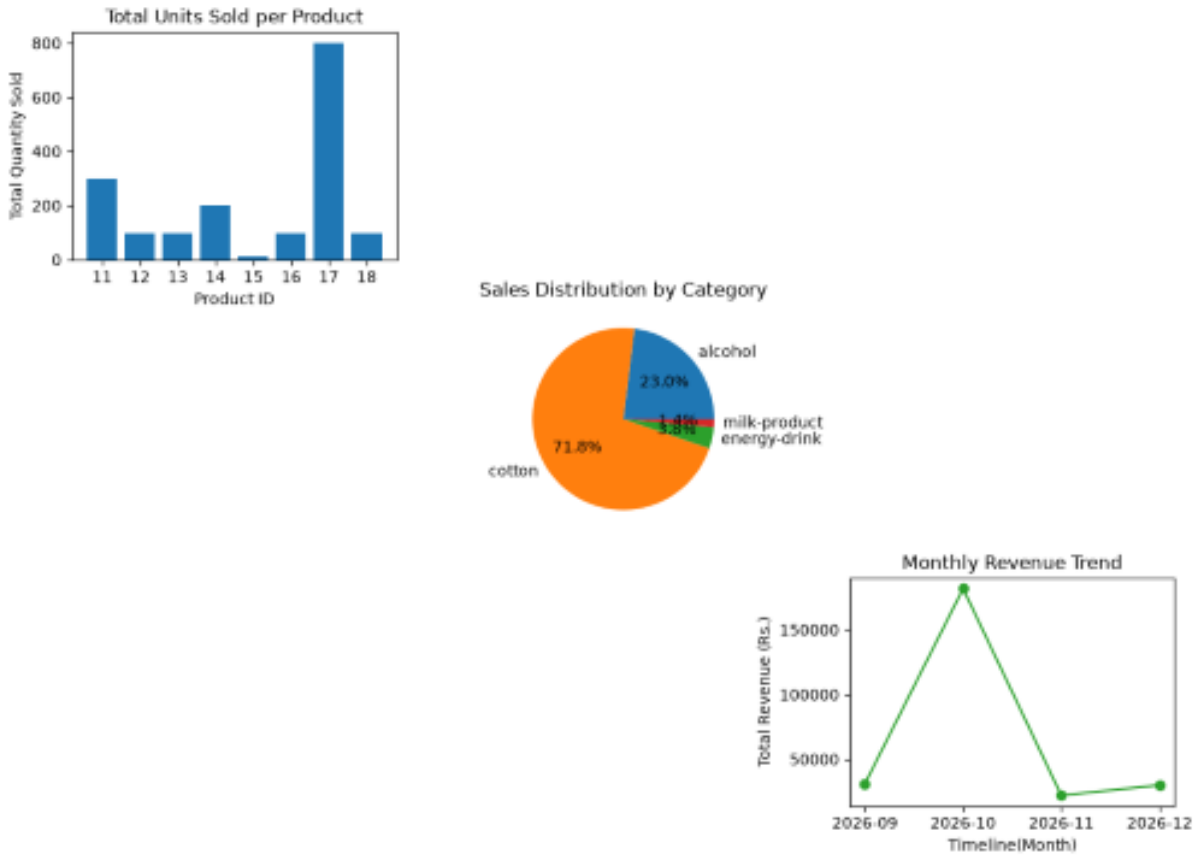


Figure 6.4: Visual Data Reports

Chapter 7

Conclusion

The Inventory and Sales Management System successfully delivers a lightweight, reliable, and user-friendly digital solution for daily store operations. By utilizing a fast command-line interface, the project proves that expensive hardware and complex graphical software are not necessary to maintain accurate business records. The system effectively automates tedious manual tasks, such as updating stock levels instantly after every transaction and calculating total revenue, significantly reducing the chances of human error in a fast-paced retail environment.

Throughout the development and testing phases, the application demonstrated strong stability and practical value. The core design securely manages products and sales, while the automated CSV file integration ensures that no critical business data is ever lost between sessions. The built-in error handling successfully protects the system from common user mistakes, such as attempting to sell out-of-stock items or duplicating product IDs, making the software highly dependable for daily use. Furthermore, the inclusion of visual data reporting elevates the project from a simple calculator to a genuine business analysis tool, turning raw sales numbers into easy-to-understand charts.

Ultimately, this project provides a strong, cost-effective foundation for small business management. While it currently operates entirely in a text-based terminal, the organized and modular code design leaves plenty of room for future upgrades, such as connecting a barcode scanner or adding a graphical interface later on. For its current scope, the software fully meets its primary goal: giving shop owners a fast, straightforward, and highly organized tool to take control of their inventory and monitor their sales with absolute confidence.

REFERENCES

[1] Python Software Foundation, *The Python Language Reference*. [Online]. Available: https://docs.python.org/3/

[2] The pandas development team, *pandas documentation*. [Online]. Available: https://pandas.pydata.org/docs/

[3] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. [Online]. Available: https://matplotlib.org/stable/index.html